Universidade Católica de Brasília
Linguagens Formais, Autômatos e Compiladores
Professor Marcelo Eustáquio
1º Semestre de 2026

RELATÓRIO TÉCNICO

Trabalho Prático — Parte 01

Analisador Léxico para a linguagem MicroPascal (μ-Pascal)

Integrantes do grupo:
Artur Afonso Pimentel Guimarães – UC19101346

Brasília — 2026

1. Introdução

Este relatório descreve a implementação de um analisador léxico para a linguagem MicroPascal (μ-Pascal), desenvolvido como parte do primeiro trabalho prático da disciplina de Linguagens Formais, Autômatos e Compiladores. O analisador foi implementado manualmente em linguagem C, sem o uso de geradores automáticos de lexer, conforme exigido pelo enunciado.

O objetivo principal é reconhecer os tokens da linguagem MicroPascal a partir de um arquivo-fonte com extensão .pas, identificar erros léxicos, manter uma Tabela de Símbolos (TS) e registrar a posição (linha e coluna) de cada token reconhecido. A implementação é baseada em um Autômato Finito Determinístico (AFD).

Ao final da execução, o programa gera três arquivos de saída: um arquivo .lex com os tokens reconhecidos, um arquivo .ts com a tabela de símbolos final e um arquivo .err com os erros léxicos encontrados.

2. Estruturas de Dados

2.1. TipoToken

Enumeração que define todos os tipos de tokens reconhecidos pelo analisador. Inclui palavras-chave (KW_PROGRAM, KW_VAR, KW_INTEGER, KW_REAL, KW_BEGIN, KW_END, KW_IF, KW_THEN, KW_ELSE, KW_WHILE, KW_DO), identificadores (ID), números inteiros (NUM_INT) e reais (NUM_REAL), operadores (OP_EQ, OP_NE, OP_LT, OP_LE, OP_GT, OP_GE, OP_AD, OP_MIN, OP_MUL, OP_DIV, OP_ASS), símbolos delimitadores (SMB_SEM, SMB_COM, SMB_OPA, SMB_CPA, SMB_COL, SMB_DOT) e os marcadores especiais TOKEN_EOF e TOKEN_ERR.

2.2. TipoErroLexico

Enumeração que categoriza os erros léxicos detectáveis: ERRO_CARACTERE_INVALIDO, ERRO_COMENTARIO_NAO_FECHADO, ERRO_STRING_NAO_FECHADA, ERRO_STRING_NAO_SUPORTADA e ERRO_NUMERO_MALFORMADO. Cada erro é associado a uma mensagem descritiva que inclui linha e coluna da ocorrência.

2.3. Token

Estrutura principal retornada pelo analisador a cada chamada. Contém os seguintes campos:

Campo	Descrição
`tipo`	Tipo do token reconhecido (TipoToken) ou TOKEN_ERR em caso de erro.
`erro`	Categoria do erro léxico (TipoErroLexico), preenchido quando tipo == TOKEN_ERR.
`lexema`	Texto normalizado do token ou o lexema inválido associado ao erro.
`linha`	Linha inicial onde o token ou erro foi encontrado no arquivo-fonte.
`coluna`	Coluna inicial onde o token ou erro foi encontrado no arquivo-fonte.

2.4. EntradaTS (Tabela de Símbolos)

Estrutura que representa uma entrada na Tabela de Símbolos, com dois campos: lexema (o texto normalizado em minúsculas) e tipo (o tipo do token associado). A TS é implementada como um vetor de EntradaTS, inicializado com as 11 palavras reservadas da linguagem. Ao encontrar um novo identificador, o analisador verifica se o lexema já existe na TS antes de inseri-lo, garantindo que não há duplicatas. Apenas palavras reservadas e identificadores válidos entram na TS.

3. Funções Implementadas

3.1. Arquivos do projeto

Arquivo	Responsabilidade
`main.c`	Ponto de entrada. Abre o arquivo-fonte, invoca o lexer em loop e grava os arquivos .lex, .ts e .err.
`lexico.h`	Header público do lexer. Define os tipos TipoToken, TipoErroLexico, Token, EntradaTS e os protótipos das funções públicas.
`lexico.c`	Implementação completa do AFD, tabela de símbolos e funções auxiliares de leitura/retrocesso de caracteres.
`afd.dot`	Diagrama Graphviz do Autômato Finito Determinístico implementado.
`Makefile`	Compilação direta com gcc -std=c99, gerando o binário lexico.

3.2. Funções de leitura e retrocesso

A função lerChar() lê o próximo caractere do arquivo-fonte ou do buffer de pushback. A função devolverChar() devolve um caractere para ser relido na próxima chamada. O mecanismo de pushback é essencial para o reconhecimento de operadores compostos (como <=, <>, >=, :=) e para determinar o fim de identificadores e números, onde o caractere seguinte ao lexema precisa ser "devolvido" ao fluxo de entrada.

3.3. Normalização

Como a linguagem MicroPascal é case-insensitive, todos os identificadores e palavras reservadas são convertidos para minúsculas durante a leitura. Assim, Program, PROGRAM e program resultam no mesmo lexema "program". Números preservam os caracteres lidos sem alteração.

3.4. Função principal do lexer

A função principal do analisador léxico é chamada repetidamente pelo main. A cada invocação, ela consome caracteres do arquivo-fonte, ignora separadores (espaços, tabulações, quebras de linha) e comentários, e retorna o próximo token reconhecido ou um token de erro. O processo termina quando TOKEN_EOF é retornado. Internamente, a função implementa o AFD descrito na seção seguinte.

3.5. Funções da Tabela de Símbolos

A inicialização da TS insere as 11 palavras reservadas (program, var, integer, real, begin, end, if, then, else, while, do). A busca na TS percorre o vetor comparando lexemas (já normalizados em minúsculas). A inserção de um novo identificador ocorre somente se o lexema ainda não existir na tabela. Lexemas inválidos nunca são inseridos.

4. Explicação do Autômato Finito Determinístico

O analisador léxico é implementado como um AFD cujos estados determinam o tipo de lexema em construção. A seguir são descritos os principais estados e suas transições.

4.1. Estado inicial (S_INICIAL)

Ignora separadores (espaço, tabulação, quebra de linha) e decide qual subautômato seguir com base no caractere lido: letras dirigem para S_ID, dígitos para S_INT, caracteres especiais para os estados de operadores ou símbolos, "{" para S_COMMENT, "'" para S_STRING e caracteres não reconhecidos geram erro de caractere inválido.

4.2. Identificadores e palavras reservadas (S_ID)

Acumula letras e dígitos até encontrar um caractere que não pertença a esse conjunto. Ao encerrar, o lexema normalizado é buscado na TS: se for uma palavra reservada, retorna o token correspondente (KW_PROGRAM, etc.); caso contrário, insere como identificador (ID) se ainda não existir na TS.

4.3. Números inteiros e reais (S_INT, S_REAL, S_REAL_E, S_REAL_ES, S_REAL_ED)

O estado S_INT reconhece sequências de dígitos (NUM_INT). Se um ponto decimal é encontrado seguido de dígito, transita para S_REAL para reconhecer a parte fracionária (NUM_REAL). Após a parte fracionária, se "E" ou "e" é encontrado, o AFD segue para S_REAL_E e, opcionalmente, S_REAL_ES (para o sinal + ou -), terminando em S_REAL_ED ao consumir os dígitos do expoente.

Casos de número malformado são tratados por estados auxiliares: tentativa de expoente diretamente sobre inteiro (1E2), expoente incompleto (1.5E), expoente com sinal sem dígitos (1.5E+) e expoente seguido de letra (1.5Ea). Nesses casos, o AFD transita para S_NUM_MAL, consome a cauda inválida até um separador e emite ERRO_NUMERO_MALFORMADO.

4.4. Operadores compostos (S_MENOR, S_MAIOR, S_DPONTOS)

O caractere "<" leva ao estado S_MENOR. Se o próximo caractere for "=", emite OP_LE; se for ">", emite OP_NE; caso contrário, devolve o caractere e emite OP_LT. Analogamente, ">" leva a S_MAIOR (OP_GE ou OP_GT) e ":" leva a S_DPONTOS (OP_ASS ou SMB_COL).

4.5. Comentários (S_COMMENT)

Ao encontrar "{", o AFD entra em S_COMMENT e consome todos os caracteres até encontrar "}". Caso o arquivo termine antes de "}", emite ERRO_COMENTARIO_NAO_FECHADO. Um "}"

encontrado fora do contexto de comentário é tratado como erro léxico (caractere inválido). Comentários aninhados não são suportados.

4.6. Strings (S_STRING)

A linguagem MicroPascal, no subconjunto adotado, não suporta strings. Ao encontrar o caractere "", o AFD entra em S_STRING e consome até encontrar o fechamento "" ou uma quebra de linha/EOF. Se a string for fechada corretamente, emite ERRO_STRING_NAO_SUPORTADA; se chegar à quebra de linha ou EOF sem fechar, emite ERRO_STRING_NAO_FECHADA. Em nenhum caso o conteúdo de string entra na TS.

5. Tokens Reconhecidos pela Linguagem

5.1. Palavras-chave

Token	Lexema	Descrição
KW_PROGRAM	program	Início do programa
KW_VAR	var	Declaração de variáveis
KW_INTEGER	integer	Tipo inteiro
KW_REAL	real	Tipo real
KW_BEGIN	begin	Início de bloco
KW_END	end	Fim de bloco
KW_IF	if	Condicional
KW_THEN	then	Cláusula then
KW_ELSE	else	Cláusula else
KW_WHILE	while	Laço de repetição
KW_DO	do	Cláusula do

5.2. Operadores

Token	Símbolo	Descrição
OP_EQ	=	Igualdade
OP_NE	<>	Diferença
OP_LT	<	Menor que
OP_LE	<=	Menor ou igual
OP_GT	>	Maior que
OP_GE	>=	Maior ou igual
OP_AD	+	Adição
OP_MIN	-	Subtração
OP_MUL	*	Multiplicação
OP_DIV	/	Divisão

OP_ASS	:=	Atribuição

5.3. Símbolos

Token	Símbolo	Descrição
SMB_SEM	;	Ponto e vírgula
SMB_COM	,	Vírgula
SMB_OPA	(	Abre parênteses
SMB_CPA	)	Fecha parênteses
SMB_COL	:	Dois-pontos
SMB_DOT	.	Ponto

6. Testes Realizados

O projeto inclui um conjunto de testes de regressão organizados no diretório testes/. Os arquivos de fixture (.lex, .ts, .err) servem como referência para validação automática do analisador.

6.1. Casos válidos (sem erros léxicos)

Arquivo	Descrição
teste1.pas	Exemplo principal do enunciado: declarações, atribuições, condicional e laço.
teste2.pas	Operadores relacionais e número real com expoente (ex.: 1.5E2, 3.0e-4).
teste3.pas	Laço while, nomes com maiúsculas/minúsculas misturadas e real simples.

6.2. Casos com erros léxicos

Arquivo	Erro testado
erro1.pas	Caractere inválido: @.
erro2.pas	Comentário não fechado antes do fim do arquivo.
erro3.pas	Múltiplos caracteres inválidos: % e $.

6.3. Exemplo de saída gerada

Para o programa do enunciado (teste1.pas), o analisador produz saída no arquivo .lex com o seguinte formato:

```
<KW_PROGRAM, program> 1 1
<ID, exemplo> 1 9
<SMB_SEM, ;> 1 16
```

```
<KW_VAR, var> 2 1
<ID, x> 3 3
<SMB_COM, ,> 3 4
<ID, y> 3 6
<SMB_COL, :> 3 8
<KW_INTEGER, integer> 3 10
<SMB_SEM, ;> 3 17
...
```

Caso não existam erros léxicos, o arquivo .err contém a mensagem: "Nenhum erro léxico encontrado."

6.4. Exemplo de Tabela de Símbolos final

Para o programa do enunciado, a TS final inclui as palavras reservadas pré-carregadas mais os identificadores encontrados:

Lexema	Tipo
`program`	Palavra reservada
`var`	Palavra reservada
`integer`	Palavra reservada
`real`	Palavra reservada
`begin`	Palavra reservada
`end`	Palavra reservada
`if`	Palavra reservada
`then`	Palavra reservada
`else`	Palavra reservada
`while`	Palavra reservada
`do`	Palavra reservada
`exemplo`	Identificador
`x`	Identificador
`y`	Identificador
`z`	Identificador

7. Formato das Mensagens de Erro

O arquivo .err gerado pelo analisador contém uma linha por erro léxico detectado, seguindo os seguintes formatos de mensagem:

```
ERRO lexico: caractere invalido 'X' na linha L, coluna C
ERRO lexico: comentario nao fechado na linha L, coluna C
ERRO lexico: string nao fechada na linha L, coluna C
ERRO lexico: string nao suportada na linha L, coluna C
ERRO lexico: numero malformado 'LEXEMA' na linha L, coluna C
```

Onde L é o número da linha e C o número da coluna em que o erro foi detectado. Se nenhum erro for encontrado, o arquivo contém apenas: "Nenhum erro léxico encontrado."

8. Políticas Léxico-Sintáticas Adotadas

8.1. Sensibilidade a maiúsculas/minúsculas

A linguagem é case-insensitive. Todos os identificadores e palavras reservadas são armazenados em minúsculas na TS. Assim, Program, PROGRAM e program são tratados como o mesmo lexema.

8.2. Comentários

Comentários utilizam o formato { ... } e são completamente ignorados pelo analisador (não geram tokens). Se o arquivo-fonte termina antes do caractere de fechamento "}", o analisador emite um erro de comentário não fechado. O caractere "}" encontrado fora de um contexto de comentário é reportado como caractere inválido. Comentários aninhados não são suportados.

8.3. Strings

Strings delimitadas por apóstrofos ('...') não fazem parte do subconjunto aceito da linguagem. Ao encontrar uma string, o analisador a consome inteiramente e emite erro de string não suportada. Se a string não for fechada antes de uma quebra de linha ou do fim do arquivo, emite erro de string não fechada. O conteúdo de strings jamais é inserido na TS.

8.4. Números

Números inteiros (NUM_INT) são sequências de um ou mais dígitos. Números reais (NUM_REAL) exigem parte inteira, ponto decimal e pelo menos um dígito após o ponto (exemplos válidos: 0.0, 2.5, 1.5E2, 3.0e-4). O expoente só é aceito após um real válido. Casos como 1E2, 1.5E, 1.5E+ e 1.5Ea são tratados como número malformado.

9. Compilação e Execução

9.1. Compilação

O projeto utiliza um Makefile que compila o código de forma direta com gcc:

```
make
```

O comando acima gera o binário executável "lexico".

9.2. Execução

Para executar o analisador sobre um arquivo-fonte:

```
./lexico testes/teste1.pas
```

Se a entrada for testes/teste1.pas, o programa gera os seguintes arquivos ao lado do fonte: testes/teste1.lex, testes/teste1.ts e testes/teste1.err.

9.3. Limpeza

```
make clean
```

Remove o executável lexico e eventuais arquivos objeto .o.

10. Limitações Conhecidas

O projeto implementa apenas a análise léxica; não verifica declaração de variáveis nem sintaxe global do programa. Comentários aninhados não são suportados. Strings são apenas diagnosticadas como erro, não existindo token de string válido. O buffer de pushback é pequeno (suficiente para o desenho atual do autômato). O analisador trata um subconjunto da linguagem Pascal conforme especificado pelo enunciado do trabalho prático.

11. Conclusão

O analisador léxico desenvolvido atende aos requisitos do trabalho prático, implementando um AFD manual em C capaz de reconhecer todos os tokens da linguagem MicroPascal, manter uma Tabela de Símbolos consistente e reportar erros léxicos com precisão de linha e coluna. A suíte de testes com casos válidos e de erro garante a cobertura dos principais cenários da especificação.